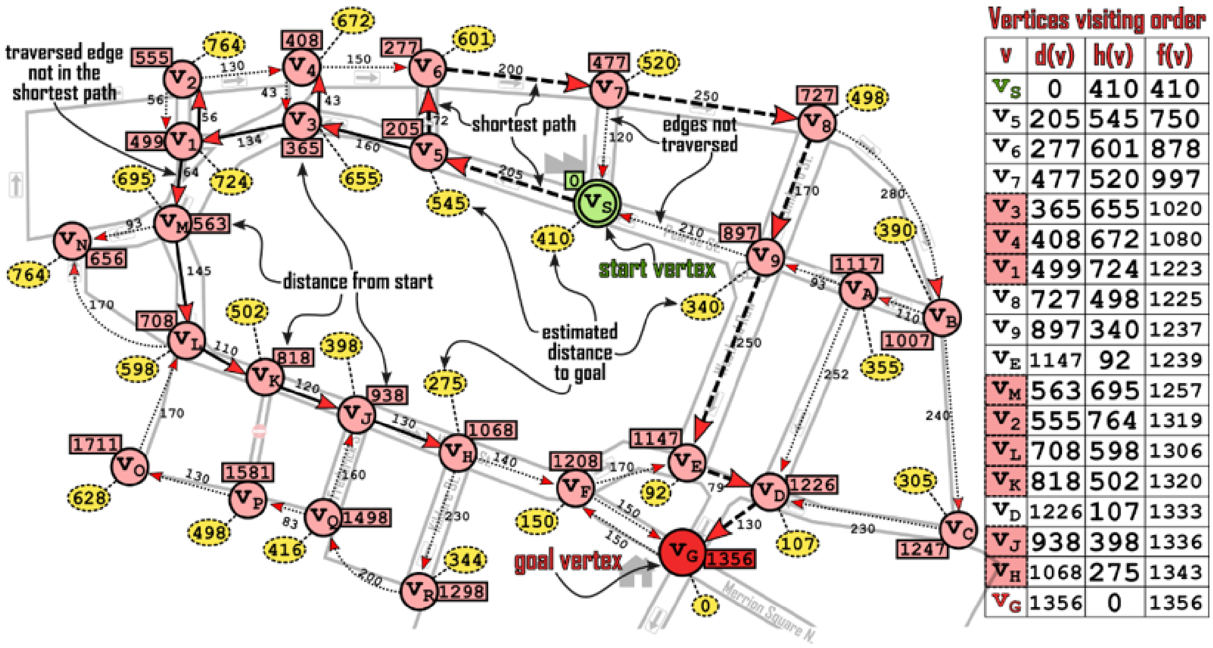
getting from v to u. In formula, `heuristic(u) ≤ distance(v, u) + heuristic(v)`.

When we plan to use A* search, the first thing we need to ensure is that we have a heuristic function that is both admissible and consistent: this condition is necessary and sufficient to ensure the optimal solution will be found[13].


traversed edge not in the shortest path
shortest path
edges not traversed
distance from start
start vertex
estimated distance to goal
goal vertex

Vertices visiting order

v	d(v)	h(v)	f(v)
v_S	0	410	410
v_5	205	545	750
v_6	277	601	878
v_7	477	520	997
v_3	365	655	1020
v_4	408	672	1080
v_1	499	724	1223
v_8	727	498	1225
v_9	897	340	1237
v_E	1147	92	1239
v_M	563	695	1257
v_2	555	764	1319
v_L	708	598	1306
v_K	818	502	1320
v_D	1226	107	1333
v_J	938	398	1336
v_H	1068	275	1343
v_G	1356	0	1356

Figure 14.22 Applying A* to the example graph in figure 14.18: while with Dijkstra's we would have had to visit all vertices whose distance from source is smaller than 1356, A* reaches the goal faster and, although it still visits some vertices not in the shortest path (highlighted with a red background in the table on the right), we can avoid paying a visit to vertices v_A, v_B, v_C, v_F, v_N, v_R.

What does all of this mean for our problem, delivering goods to customers? Well, to speed-up the search for the best